

Accuracy Report: Safe Coding-Project Scanner v1.0.0

All results below come from actual scanner runs (`python -m scanner scan <fixture> --output json`), not hand predictions. The pytest suite (49 tests) asserts these results, so a regression fails CI.

Legend: TP = expected and detected · TN = correctly not detected · FP = detected but benign or not expected · FN = expected but missed.

1. Fixture results

Rule ID	Test Fixture	Expected	Detected	Result	Notes
(all rules)	clean-project	No findings	No findings	TN	Overall: CLEAN, score 0, 7 files
VSCODE_AUTO_TASK	vscode-task-project	Yes (HIGH)	Yes, <code>.vscode/tasks.json:11</code>	TP	Only finding. Score 25, level MEDIUM
ENCODED_CONTENT	encoded-script-project	Yes (MEDIUM)	Yes, <code>en-coded_loader.py:9</code>	TP	Triggered by the <code>b64decode()</code> call. The 48-character blob is below the 60-character long-string threshold
DYNAMIC_EVAL	encoded-script-project	Yes (MEDIUM)	Yes, <code>en-coded_loader.py:9</code>	TP	Score 20, level LOW
PKG_INSTALL_SCRIPT	install-hook-project	Yes (HIGH)	Yes, <code>package.json:7</code>	TP	<code>postinstall</code> hook. Score 25, level MEDIUM
NETWORK_IN_SETUP	install-hook-project	No (no <code>setup.py</code>)	No	TN	Hook only runs <code>echo</code> . The network regex does not match
PKG_INSTALL_SCRIPT	multi-warning-project	Yes	Yes, <code>package.json:6</code>	TP	
VSCODE_AUTO_TASK	multi-warning-project	Yes	Yes, <code>.vscode/tasks.json:9</code>	TP	
ENCODED_CONTENT	multi-warning-project	Yes	Yes, <code>payload.py:5</code>	TP	Long base64 literal (≥ 60 characters) on line 5

Rule ID	Test Fixture	Expected	Detected	Result	Notes
DYNAMIC_EVAL	multi-warning-project	Yes	Yes, <code>payload.py:9</code>	TP	
CREDENTIAL_ACCESS	multi-warning-project	Yes (CRITICAL)	Yes, <code>setup.py:8</code>	TP	<code>~/.ssh/id_rsa</code> string
NETWORK_IN_SETUP	multi-warning-project	Yes (HIGH)	Yes, <code>setup.py:6</code>	TP	<code>import requests</code> . <code>requests.get</code> is on line 13; only the first match is reported
(composite)	multi-warning-project	CRITICAL	CRITICAL (score 145)	TP	Composite rule triggered

Fixture totals: 11 TP, 0 FP, 0 FN; clean-project and install-hook-project network check were true negatives.

2. Rule-level unit cases (tmp_path SIMULATION repos)

Rules not covered by the five fixtures each have a simulated positive case in `tests/test_scanner.py::test_rule_detects_simulated_case`.

Rule ID	Simulated input	Detected	Result
SETUP_PY_CMDCLASS	<code>setup(cmdclass={'install': X})</code>	Yes	TP
VSCODE_SETTINGS_EXEC	<code>terminal.integrated.defaultProfile.linux</code>	Yes	TP
SHELL_IN_SETUP	<code>subprocess.run(...)</code> in <code>setup.py</code>	Yes	TP
OBFUSCATED_JS	<code>eval(atob('...'))</code>	Yes	TP
CREDENTIAL_ACCESS	SSH key path in a <code>.py</code> file	Yes	TP
NETWORK_IN_SETUP	<code>import urllib.request</code> in <code>setup.py</code>	Yes	TP
UNUSUAL_DEPENDENCY_URL	<code>git+http://example.invalid/...</code> in <code>requirements.txt</code>	Yes	TP
MAKEFILE_PHONY	<code>all: + tab recipe</code>	Yes	TP
CI_SECRETS_EXPOSURE	<code>echo \${{ secrets.X }} > leak.txt</code>	Yes	TP
POLYGLOT_FILE	<code>.png</code> containing <code><script></code>	Yes	TP
HIDDEN_EXECUTABLE	executable <code>.helper</code> with shebang	Yes	TP

3. False-positive and false-negative probes

These are benign or evasive inputs scanned on purpose to measure the rules' limits:

Scenario	Rule(s) fired	Result	Notes
husky "prepare": "husky install"	PKG_INSTALL_SCRIPT	FP (by design)	Legitimate. Flagged because any automatic hook needs a human to read it
"postinstall": "node scripts/download-binary.js" (esbuild/puppeteer style, as in many create-react-app dependency trees)	PKG_INSTALL_SCRIPT	FP (by design)	Same reasoning. Network use inside the called JS file is not followed (see limitations)
re-require('dotenv').config({ path: '.env' })	CREDENTIAL_ACCESS	FP	Very common legitimate pattern, but rated CRITICAL. The biggest FP source
echo \${ secrets.DOCKER_PASSWORD } \ docker login --password-stdin	CI_SECRETS_EXPOSURE	FP	Standard Docker login idiom
Makefile all: → gcc -o app main.c	MAKEFILE_PHONY	FP (by design)	Nearly every C project matches. MEDIUM means "review", not "malicious"
Embedded PNG icon as base64 string in .py	ENCODED_CONTENT	FP	Expected. The reviewer decodes it and sees image bytes
--extra-index-url https://pkgs.example.invalid/simple	UNUSUAL_DEPENDENCY_URL	TP	Dependency-confusion risk. Private registries will also match
SSH path built by concatenation: "." + "ss" + "h"	none	FN	String splitting defeats the keyword rules
import-lib.import_module("req" + "uests") in setup.py	none	FN	Dynamic imports hide network access

Scenario	Rule(s) fired	Result	Notes
<code>process.env.PORT</code> , <code>re.compile(...)</code> , <code>cursor.exec(...)</code>	none	TN	Guarded by lookbehinds ((?<![\w.]))
Valid PNG header with no script markers	none	TN	

4. Rules adjusted during testing

- **DYNAMIC_EVAL:** added the lookbehind ((?<![\w.])) so method calls such as `cursor.exec()` and `regex.exec()` do not match. Comment lines are skipped. Files under `test/` , `tests/` , `__tests__/` , `spec/` and files named `test_*.py` are excluded, as the spec requires.
- **CREDENTIAL_ACCESS:** the `.env` pattern now requires a path-like boundary (quote, slash, whitespace, `=` , `(`) , so `process.env` and `os.environ` no longer match. CI workflow files are excluded because `secrets.GITHUB_TOKEN` is normal there. `CI_SECRETS_EXPOSURE` handles misuse in those files instead.
- **ENCODED_CONTENT:** long-base64 matches must contain digits, upper case and lower case, so long identifiers and paths don't match. Bare hex needs 100+ characters so SHA-256 digests (64 characters) don't match. `base64 -d` is only checked in shell scripts. This stopped the remediation text from matching itself.
- **NETWORK_IN_SETUP:** for `package.json` , only the install lifecycle script values are checked, so a `build` script using `curl` is not flagged.
- **UNUSUAL_DEPENDENCY_URL:** for `package.json` , only dependency sections are parsed, so `homepage` and `repository` URLs no longer match.
- **CI_SECRETS_EXPOSURE:** `curl` was removed from the “write” verbs, because sending a secret in an auth header to an API is normal.
- **Composite rule:** the HIGH/CRITICAL finding must be a different finding from the credential or network trigger. Otherwise a single `CREDENTIAL_ACCESS` finding would trigger itself. Two install-time behaviours (for example a preinstall hook plus `urllib` in `setup.py`) still escalate to CRITICAL even though their score is only 50.
- **Self-scan hygiene:** keyword regexes use one-character classes (for example `id_rs[a]`) so the scanner's own source does not match itself. A self-scan with `--exclude tests/fixtures` returns **CLEAN** (0 findings), which the CI gate requires.

5. Known false-positive scenarios

- Legitimate `postinstall` / `prepare` hooks (husky, node-gyp native builds, esbuild/puppeteer/electron binary downloaders, packages in create-react-app dependency trees).
- `dotenv` loaders that reference `.env` . These are rated CRITICAL, so always check context.
- Teams that auto-start a dev server with a `folderOpen` task.
- C extension builds that customise `build_ext` (`SETUP_PY_CMDCLASS` / `SHELL_IN_SETUP`).
- Private corporate registries (Artifactory, Nexus) flagged by `UNUSUAL_DEPENDENCY_URL`.
- `docker login --password-stdin` piping in CI.
- Base64-embedded fonts, icons and test vectors.

6. Limitations

- Static scanning cannot detect every threat.
- Obfuscated behaviour may evade rules (string splitting, dynamic imports and multi-stage loaders were shown above to be false negatives).
- Legitimate developer tools sometimes resemble malicious behaviour (see section 5).
- A clean result is not a guarantee of safety.
- Dynamic analysis should occur only in a controlled sandbox.

Additional engineering limits: the scanner reports only the first match per rule per file. It does not follow `postinstall` into the scripts that hook calls. Files over 2 MB are only checked by binary-safe rules. Minified or bundled JS is not deobfuscated.